Field Intelligence App for Native Plants

Implementation Guide — Teammate 2: AR & Frontend Lead

GEG XR Hackathon — Native or Web Application, Android Build

Scope covered in this guide: App Architecture, Camera Scanner Interface, AR Visualization (species metadata overlay), and Chat Interface Integration with the AI Botanical Guide API.

Companion deliverable: a complete Android Studio project (Kotlin + Jetpack Compose) named **FieldIntelligenceApp**, provided alongside this PDF as a .zip you can unzip and open directly in Android Studio.

1. What this delivers

This package implements everything listed under your role on the team slide:

- **App Architecture** — a native Android app (Kotlin + Jetpack Compose, minSdk 26) with a clean MVVM structure: UI screens, ViewModels, a Retrofit-based data layer, and a CameraX camera module.
- **AR Visualization** — a live camera scanner screen (CameraX) and an AR-style overlay screen that displays species metadata (Scientific Name, Family, Native Region, Conservation Status, native/non-native flag, confidence, description) as a translucent card anchored over the camera feed.
- **Chat Interface Integration** — a full chat UI (message bubbles, input bar, loading state) wired to a */chat* endpoint on the AI Botanical Guide backend, optionally grounded in the last scanned species.

It is built so it keeps working at the hackathon table even if your AI/backend teammate's API isn't finished yet: both the plant-ID call and the chat call fall back to sensible local demo data / messages if the network request fails, instead of crashing.

2. Why Kotlin + Jetpack Compose (not Unity/WebXR)

The team slide lists several valid options: Unity + AR Foundation, WebXR/MindAR, React Native, or Flutter. Since you asked specifically for an **Android app**, this guide uses native Android (Kotlin + Jetpack Compose + CameraX + ARCore) because it:

- Produces a real installable .apk with no extra runtime (no WebView/JS bridge, no Unity export step).
- Has first-class CameraX support for the scanner, and ARCore for true world-anchored overlays if you want to extend past the screen-space overlay included here.
- Is the fastest path from this codebase to a signed APK for your working-prototype submission.

If your team ultimately builds the AR core in Unity instead, everything in Sections 4–6 below (the API contract, the metadata fields, and the chat UX) still applies — only the rendering layer changes.

3. Project structure

```
FieldIntelligenceApp/
  settings.gradle.kts, build.gradle.kts, gradle.properties
  app/build.gradle.kts            <- dependencies (CameraX, ARCore, Retrofit, Compose)
  app/src/main/AndroidManifest.xml
  app/src/main/java/com/geg/fieldintel/
    MainActivity.kt               <- entry point, sets Compose content
    ui/AppNavGraph.kt             <- navigation: Scanner -> AR Result -> Chat
    ui/scanner/ScannerScreen.kt   <- CameraX preview + capture button
    ui/scanner/ScannerViewModel.kt
    ui/arresult/ARResultScreen.kt <- AR metadata overlay card
    ui/chat/ChatScreen.kt         <- chat UI
    ui/chat/ChatViewModel.kt
    ui/theme/Theme.kt             <- app color palette (forest greens)
    camera/CameraXController.kt   <- camera lifecycle + photo capture
    data/model/Species.kt         <- Species, ConservationStatus, ScanResult
```

```
    data/model/ChatMessage.kt
    data/remote/BotanicalGuideApi.kt   <- Retrofit interface (/identify, /chat)
    data/remote/RetrofitClient.kt
    data/repository/PlantRepository.kt <- API + offline demo fallback, 7 seed species
    data/repository/ChatRepository.kt
  backend-example/main.py         <- optional FastAPI reference backend using Claude
```

4. Setting up the project

4.1 Requirements

- Android Studio (Koala/2024.1 or newer)
- JDK 17 (bundled with recent Android Studio)
- An Android device or emulator running API 26+ (API 24+ recommended to also have Google Play Services for AR)

4.2 Open and run

- Unzip **FieldIntelligenceApp.zip**.
- Open the folder in Android Studio (File → Open) and let Gradle sync.
- Connect a physical device (recommended — camera + AR work far better than the emulator) or start an emulator with a virtual camera enabled.
- Click Run. Grant the camera permission when prompted.

4.3 Point the app at your backend

Open **app/build.gradle.kts** and change this line to your team's real API URL:

```
buildConfigField("String", "BOTANICAL_API_BASE_URL", "\"https://your-backend.example.com/\""
)
```

For local testing against the included example backend running on your laptop: use `http://10.0.2.2:8000/` from the emulator, or `http://<your-laptop-LAN-IP>:8000/` from a physical phone on the same Wi-Fi.

5. The API contract (your integration boundary with the AI/Backend teammate)

The app talks to two endpoints, defined in `BotanicalGuideApi.kt`. Share this contract with whoever owns the AI/model side so both halves can be built in parallel.

POST /identify (multipart/form-data)

```
Request:  image=<jpeg file>, lat=<optional>, lng=<optional>
Response: {
  "status": "success" | "multiple_candidates" | "no_match" | "error",
  "species": {
     "id", "commonName", "scientificName", "family", "nativeRegion",
     "conservationStatus": "LC|NT|VU|EN|CR|EW|NE|DD",
     "isNative": true/false, "shortDescription", "funFact",
     "imageUrl", "confidence": 0.0-1.0
  }
}
```

POST /chat (application/json)

```
Request:  { "message": string, "speciesId": string|null, "conversationId": string|null }
Response: { "reply": string, "conversationId": string }
```

A working reference implementation of both endpoints (using the Anthropic API for vision identification and conversational replies) is included in **backend-example/main.py**. It's a starting point, not a requirement — any backend that satisfies this JSON contract will work with the Android app unchanged.

6. How the AR visualization works

ScannerScreen shows a live CameraX preview with a square viewfinder guide. Tapping the shutter button captures a JPEG, hands it to `PlantRepository.identify()`, which uploads it to `/identify`.

ARResultScreen keeps the camera feed running behind a translucent metadata card pinned to the bottom of the screen — this reads as an AR heads-up display without requiring full ARCore plane tracking, which keeps the demo reliable on any device. The card shows:

- Common name + a colored conservation-status badge (green = Least Concern/Near Threatened, amber = Vulnerable, red = Endangered/Critically Endangered/Extinct in the Wild)
- Scientific name (italicized)
- Family, Native Region, native/non-native flag, and match confidence
- A short description and an optional fun fact
- An "Ask the AI Botanical Guide about this plant" button that jumps into the chat, pre-grounded in that species' id

Upgrading to true world-anchored AR (optional, for extra AR/AI Integration points)

To anchor the card to the physical plant instead of the screen edge, add ARCore's Session + plane detection: create an `com.google.ar.core.Session`, run hit-tests on tap, create an `Anchor` at the hit pose, and render the Compose overlay at that anchor's projected screen coordinates each frame (or use SceneView / Filament for full 3D-anchored UI). The `com.google.ar:core` dependency is already declared in `app/build.gradle.kts` so you can build this out incrementally without changing project setup.

7. Chat interface integration

`ChatScreen.kt` renders a standard bubble-style chat: user messages right-aligned in the primary color, assistant replies left-aligned, with an animated "Thinking…" state while a request is in flight. `ChatViewModel` keeps message history in a `StateFlow` and calls `ChatRepository.sendMessage()`, which posts to `/chat` and carries a `conversationId` across turns so the backend can maintain context.

When a user taps "Ask the AI Botanical Guide about this plant" on the AR overlay, the chat opens with that species' id pre-attached, so the backend can ground its answer (e.g. "Why is this species vulnerable?") without the user re-explaining what they scanned.

8. Fitting this into the full team submission

Per the challenge brief, your team still needs, beyond this AR/frontend module:

- At least **7 native/campus plant species** with real data — replace the 7 seed entries in `PlantRepository.DemoSpeciesData` with your team's actual campus survey data.
- A trained/working species-ID model or vision pipeline behind `/identify` (the AI/Data teammate's responsibility — `backend-example/main.py` is a quick way to get a working version using Claude's vision understanding).
- The **presentation deck** (5–6 slides) with screenshots of this app, a demo video, and documented feedback from at least 5 users.
- Optional innovation features called out in the brief — e.g. rare/endangered-species highlighting (already wired via the conservation-status badge colors), gamified discovery, offline inference, and species comparison — can be layered onto this codebase; the AR/chat/scanner scaffolding is built to accept them without a rewrite.

9. Troubleshooting

- **Black camera preview:** confirm the camera permission was granted and you're running on a physical device or an emulator with a working virtual camera.
- **Network calls always fall back to demo data:** your `BOTANICAL_API_BASE_URL` is unreachable — check it's the correct LAN IP/URL and that `usesCleartextTraffic` is enabled in the manifest if you're testing over plain HTTP.
- **Gradle sync errors:** make sure Android Studio is using JDK 17 (File → Project Structure → SDK Location).